

Problem Solving — টপ ৫ প্রবলেম

সিনিয়র লেভেল রিভিশন — ১৫ মিনিট। এই ৫টা প্যাটার্ন (hash map, two pointer, sliding window, stack, top-K) জানা থাকলে ইন্টারভিউয়ের বেশিরভাগ মাঝারি প্রবলেম এগুলোরই ভ্যারিয়েশন। ভাষা: PHP-য়েঁষা ইন্টারভিউ হলে PHP-তেই লেখা নিরাপদ — নিচে PHP ব্যবহার করা হয়েছে।

১. Two Sum (Hash Map প্যাটার্ন)

Description: একটা array আর একটা target দেওয়া — এমন দুইটা সংখ্যার index বের করতে হবে যাদের যোগফল target। সবচেয়ে জিজ্ঞাসিত ওয়ার্ম-আপ; আসল পরীক্ষা: brute force $O(n^2)$ থেকে hash map $O(n)$ -এ আনতে পারা।

কোড:

```
function twoSum(array $nums, int $target): array
{
    $seen = []; // value => index

    foreach ($nums as $i => $num) {
        $need = $target - $num;

        if (isset($seen[$need])) {
            return [$seen[$need], $i];
        }

        $seen[$num] = $i;
    }

    return [];
}
```

ব্যাখ্যা: প্রতিটা সংখ্যায় এসে দেখি "আমার জোড়া ($\$target - num$) কি আগে দেখা গেছে?" — hash map-এ $O(1)$ lookup। না পেলে নিজেকে map-এ রেখে এগোই। এক পাসেই শেষ, sort লাগে না, index-ও অক্ষত থাকে। **Time: $O(n)$** (এক পাস, প্রতি ধাপে $O(1)$ lookup) | **Space: $O(n)$** (worst case সব element map-এ)।

২. Valid Parentheses (Stack প্যাটার্ন)

Description: " $\{[()]\}$ " জাতীয় string-এ bracket-গুলো সঠিকভাবে খোলা-বন্ধ হয়েছে কিনা চেক। Stack-এর ক্লাসিক ব্যবহার — nested structure মানেই stack, এই intuition যাচাই করা হয়।

কোড:

```
function isValid(string $s): bool
{
    $stack = [];
    $pairs = ['(' => ')', '[' => ']', '{' => '}'];

    for ($i = 0; $i < strlen($s); $i++) {
```

```

    $ch = $s[$i];

    if (!isset($pairs[$ch])) {
        $stack[] = $ch;           // opening bracket – push
    } elseif (array_pop($stack) !== $pairs[$ch]) {
        return false;           // closing – মিলল না বা stack খালি
    }

    return empty($stack);       // সব খোলাই বন্ধ হয়েছে কিনা
}

```

ব্যাখ্যা: খোলা bracket stack-এ জমাই; বন্ধ bracket এলে stack-এর মাথায় ঠিক তার জোড়া থাকতেই হবে (nested নিয়ম) — না থাকলে invalid। শেষে stack খালি না থাকলে কিছু bracket খোলা রয়ে গেছে। `array_pop` খালি stack-এ `null` দেয়, তাই আলাদা `empty`-চেক লাগে না — তুলনাতেই fail করে। **Time: $O(n)$ | Space: $O(n)$** (worst case সব opening — যেমন `"((((("`)।

৩. Longest Substring Without Repeating Characters (Sliding Window প্যাটার্ন)

Description: একটা string-এ repeat-না-করা সবচেয়ে লম্বা substring-এর দৈর্ঘ্য। Sliding window-র সেরা উদাহরণ — nested loop $O(n^2)$ -কে দুই pointer-এ $O(n)$ করা যায় কিনা সেটাই পরীক্ষা।

কোড:

```

function lengthOfLongestSubstring(string $s): int
{
    $lastSeen = []; // char => সর্বশেষ index
    $best = 0;
    $left = 0;      // window-র বাম প্রান্ত

    for ($right = 0; $right < strlen($s); $right++) {
        $ch = $s[$right];

        // duplicate window-র ভেতরে থাকলে বাম প্রান্ত তার পরে সরেও
        if (isset($lastSeen[$ch]) && $lastSeen[$ch] >= $left) {
            $left = $lastSeen[$ch] + 1;
        }

        $lastSeen[$ch] = $right;
        $best = max($best, $right - $left + 1);
    }

    return $best;
}

```

ব্যাখ্যা: `[left, right]` window সবসময় duplicate-মুক্ত থাকে। ডান প্রান্ত এগোতে এগোতে duplicate পেলে বাম প্রান্ত লাফিয়ে duplicate-এর পরের ঘরে যায় (এক ঘর করে নয় — `lastSeen` map-এর কারণে)। প্রতিটা pointer সর্বোচ্চ একবারই পুরো string পার হয়। **Time: $O(n)$ | Space: $O(\min(n, \text{charset}))$** — bounded charset-এ কার্যত $O(1)$ ।

8. Merge Intervals (Sort + Sweep প্যাটার্ন)

Description: Overlapping interval-গুলো merge করা — $[[1, 3], [2, 6], [8, 10]] \rightarrow [[1, 6], [8, 10]]$ ।
বাস্তব কাজের সবচেয়ে কাছের প্রবলেম (মিটিং শিডিউল, ডেলিভারি টাইম-স্লট, রিপোর্টের date range) — তাই সিনিয়রদের প্রিয়।

কোড:

```
function mergeIntervals(array $intervals): array
{
    if (empty($intervals)) {
        return [];
    }

    usort($intervals, fn ($a, $b) => $a[0] <=> $b[0]); // start অনুযায়ী sort

    $merged = [$intervals[0]];

    foreach (array_slice($intervals, 1) as $current) {
        $last = &$merged[count($merged) - 1];

        if ($current[0] <= $last[1]) {
            $last[1] = max($last[1], $current[1]); // overlap – শেষ প্রান্ত
        } else {
            $merged[] = $current; // gap – নতুন interval
        }
    }

    return $merged;
}
```

ব্যাখ্যা: Start অনুযায়ী sort করলে overlap শুধু পাশের interval-এর সাথেই সম্ভব — এটাই মূল insight। তারপর এক পাস: current-এর শুরু যদি শেষ merged-এর শেষের ভেতরে ($\leq$) পড়ে, দুটো মিশাই (end = max দুটোর); নাহলে নতুন গ্রুপ শুরু।
max जरুরি কারণ $[1, 10], [2, 3]$ -এ current-এর end ছোটও হতে পারে। **Time: $O(n \log n)$** (sort-ই dominant, sweep $O(n)$) | **Space: $O(n)$** (output)।

৫. Top K Frequent Elements (Hash Map + Heap/Bucket প্যাটার্ন)

Description: Array-তে সবচেয়ে বেশি বার আসা K-টা element। দুই ধাপ চিন্তা যাচাই হয়: frequency গোনা (hash map) + সেরা K বাছা (heap/bucket) — "টপ N প্রোডাক্ট/কাস্টমার" রিপোর্টের ছোট সংস্করণ।

কোড:

```
function topKFrequent(array $nums, int $k): array
{
    $freq = array_count_values($nums); // value => count, O(n)

    // Bucket sort: index = frequency, value = সেই frequency-র element-রা
    $buckets = [];
```

```

    foreach ($freq as $value => $count) {
        $buckets[$count][] = $value;
    }

    $result = [];
    for ($count = count($nums); $count >= 1 && count($result) < $k; $count--) {
        if (!isset($buckets[$count])) {
            continue;
        }
        foreach ($buckets[$count] as $value) {
            $result[] = $value;
            if (count($result) === $k) {
                break;
            }
        }
    }

    return $result;
}

```

ব্যাখ্যা: Frequency গুনে (hash map) elements-দের frequency-নম্বর bucket-এ ফেলি — frequency সর্বোচ্চ n হতে পারে, তাই bucket array bounded। তারপর উপরের bucket থেকে নামতে নামতে K -টা তুললেই শেষ — sort ছাড়াই। বিকল্প উত্তর জানা রাখা: min-heap of size $K \rightarrow O(n \log k)$ — stream/বিশাল ডেটায় সেটা ভালো; ইন্টারভিউতে দুটোর তুলনা বললে বাড়তি নম্বর। **Time: $O(n)$** (গোনা + bucket সুইপ) | **Space: $O(n)$** (map + buckets)।

দ্রুত রিভিশন চিটশিট (৩০ সেকেন্ডে)

প্যাটার্ন	সংকেত (কখন ব্যবহার)	Complexity
Hash Map	"জোড়া/complement খোঁজা", "আগে দেখেছি কিনা"	$O(n)$
Stack	Nested/matching structure, "সর্বশেষ খোলাটা আগে বন্ধ"	$O(n)$
Sliding Window	Contiguous substring/subarray-র max/min	$O(n)$
Sort + Sweep	Interval/range overlap, schedule	$O(n \log n)$
Bucket / Heap	Top-K, frequency-ভিত্তিক বাছাই	$O(n) / O(n \log k)$